

Concurrency & Distributed Systems

Part 2 — Scaling, Limits, and Engineering Judgment

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

From Sequential to Concurrent

Single-threaded problem:

- One request blocks all others
- Latency grows linearly under burst load

Solution: Thread Pool

```
ExecutorService pool =  
    Executors.newFixedThreadPool(10);  
  
pool.execute(() -> handleRequest(client));
```

What you should observe

Latency collapses under load. Throughput increases. CPU usage increases.

STUDENT TASK 1 — Run the Multi-Threaded Server

Goal: Compare single-thread vs thread-pool server.

Terminal A (server):

```
javac MultiThreadedServer.java  
java MultiThreadedServer 10
```

Terminal B (client):

```
javac LoadTestClient.java  
java LoadTestClient --clients=50 --requests  
=20 --warmup=20 --csv=pool10.csv
```

Record: p50, p95, throughput (req/s).

STUDENT TASK 2 — Sweep Pool Size

Run the same load with different pool sizes.

```
java MultiThreadedServer 5
java LoadTestClient --clients=50 --requests
    =20 --warmup=20 --csv=pool5.csv

java MultiThreadedServer 10
java LoadTestClient --clients=50 --requests
    =20 --warmup=20 --csv=pool10.csv

java MultiThreadedServer 50
java LoadTestClient --clients=50 --requests
    =20 --warmup=20 --csv=pool50.csv
```

Question: Does bigger pool always improve p95?

LoadTestClient — Turning Lecture into Data

We measure:

- **p50 latency** (median)
- **p95 latency** (tail latency)
- **Throughput** (requests/sec)

Why p95 matters:

- Users remember the slowest requests
- Tail latency dominates perceived performance

Key Shift

We no longer guess. We measure.

Fill This Table (Required)

Pool Size	p50 (ms)	p95 (ms)	Throughput (req/s)
5			
10			
50			

Submission rule

Your table must be supported by your CSV files.

Replace I/O with CPU

Instead of (I/O wait):

```
Thread.sleep(100);
```

We run (CPU work):

```
int result = fibonacci(35);
```

Now repeat the same load test.

STUDENT TASK 3 — CPU-Bound Experiment

Terminal A (server):

```
javac CPUBoundServer.java  
java CPUBoundServer 10 35
```

Terminal B (client):

```
java LoadTestClient --clients=50 --requests  
=10 --warmup=20 --csv=cpu_pool10.csv
```


STUDENT TASK 3 — CPU-Bound Experiment

Terminal A (server):

```
javac CPUBoundServer.java  
java CPUBoundServer 10 35
```

Terminal B (client):

```
java LoadTestClient --clients=50 --requests  
=10 --warmup=20 --csv=cpu_pool10.csv
```

Now increase threads:

```
java CPUBoundServer 50 35  
java LoadTestClient --clients=50 --requests  
=10 --warmup=20 --csv=cpu_pool50.csv
```

Question: Did p95 improve or degrade? Why?

What Happens Now?

Expected observations:

- CPU reaches 100%
- Throughput plateaus (limited by cores)
- Large pools can **increase** p95 (context switching)

Core idea

Concurrency is not magic speed. CPU-bound workloads are limited by hardware.

Amdahl's Law (Simplified)

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

CPU-bound workload:

$$P \approx 1$$

Speedup limited by:

$$S_{max} \approx \text{number of cores}$$

Conclusion

Threads do not create CPU power.

$$\text{OptimalThreads} = \text{Cores} \times \left(1 + \frac{\text{Wait}}{\text{Compute}} \right)$$

Interpretation:

- Heavy I/O $\rightarrow$ larger pools are useful
- CPU-bound $\rightarrow$ pool near core count

STUDENT TASK 4 — Use ThreadPoolTuner

Run:

```
javac ThreadPoolTuner.java  
java ThreadPoolTuner
```

Write in your report:

- Your machine core count
- Recommended pool size for (90/10), (50/50), (10/90)
- Compare recommendation with your best measured pool size

What You Must Submit

Deliverables (Lab 1.1):

- A filled results table (pool size vs p50/p95/throughput)
- CSV files generated by LoadTestClient
- CPU-bound comparison: pool=10 vs pool=50 (explain p95 change)
- One paragraph: connect your results to Amdahl + tuning formula

Minimum acceptable report

If you have no CSV evidence, your numbers do not count.

If threads become expensive and cores limit CPU work, what architecture avoids large thread pools entirely?

Next Topic: Event-Driven / Async I/O